

Pact Governance Audit Report

Table of Contents

1. Executive Summary	4
About Pact Governance Protocol	4
Risk Profile	4
Overall Security Posture	4
After-fix Review	4
Before-fix Review	4
Audit Scope	5
2. Assumptions and Considerations	6
Audit Assumptions	6
Centralization Risks	6
3. Severity Definitions	7
Impact	7
Likelihood	7
Severity Classification Matrix	7
4. Findings	9
M01: Incorrect issuance rate when ammend_vesting_end is used before the cliff	9
M02: Locks are not processed in order as expected	10
M03: Unlock history can grow unbounded	12
L01: State bloat via unauthenticated vesting initialization	14
L02: Integer underflow leads to DoS in vesting::claim	15
L03: RewardsClaimed event is always emitted with epochs_processed==0	16
L04: The loop in unsynced_user_epoch_progress can run out of gas	17
L05: global_vesting_amounts is never updated so global_vested_by_schedule is wrong	19
L06: Airdrop can be created already expired	21
L07: Admin can withdraw remaining even when the airdrop has not expired.	23
L08: Zero-amount unlocks are accepted	25

5. Enhancement Opportunities

27

E01: Enforce more checks in ammend_vesting_end

27

E02: Enforce more checks in update_user_schedule

28

E03: withdraw_remaining ignores EINSUFFICIENT_BALANCE and balance checks

29

E04: Fragile unlock logic

30

E05: Incorrect error code in get_upgrader_signer

32

E06: Assert before voting power truncation (u128 to u64)

33

About Us

34

About Adevar Labs

34

Audit Methodology

34

Confidentiality Notice

35

Legal Disclaimer

35

1. Executive Summary

About Pact Governance Protocol

The Pact Governance Protocol is a comprehensive governance and token distribution infrastructure built on Aptos and written in Move, providing modular vesting schedules, vote-escrowed token mechanics, reward distribution capabilities, and auction-based buyback systems. The protocol implements time-locked voting power, merkle-proof-based airdrops, epoch-based reward tracking, and governance parameter management for sophisticated tokenomics design. This audit focused on the governance, vesting, veToken, airdrop, distribution, and buyback modules, examining authorization controls, time-based calculation logic, state management patterns, and potential edge cases in schedule amendments and fund handling.

Risk Profile

The following table summarizes the distribution of identified vulnerabilities by risk level:

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	-
High	0	0	-
Medium	3	3	-
Low	8	8	-

Number of enhancement opportunities: 6

Overall Security Posture

After all issues were addressed, the protocol now reflects a clean and well-engineered design, with clear validation paths, consistent state transitions, and robust safeguards across vesting, governance, and distribution flows. The fixes further refined an already solid architecture, tightening edge-case handling and strengthening authorization boundaries without altering the underlying structure. The result is a cohesive, reliable system that aligns closely with Move best practices and is well-prepared for secure production use.

After-fix Review

After the initial audit, all reported issues have been fully addressed with appropriate fixes across the codebase. The team added the missing authentication checks, corrected vesting-schedule invariants, resolved issuance-rate edge cases, removed underflow conditions, enforced existence checks on schedule updates, and ensured that vesting, locking, and unlocking logic behaves consistently and safely under all scenarios. Event data is now emitted accurately, state growth vectors have been capped, error codes have been corrected, and previously missing expiry and balance validations in the airdrop flow have been implemented. Together, these changes resolve the identified weaknesses and bring the protocol to a more robust and reliable state suitable for production deployment.

Before-fix Review

This initial audit report presents 11 findings for the team's consideration. We recommend that the development team prioritize remediation based on severity:

Medium-severity findings should be addressed first to enhance protocol robustness. These include issues across vesting calculations (incorrect issuance rates and integer underflows), auction mechanics, state management, and access control.

Low-severity findings represent opportunities for improved validation, event accuracy, and configuration safeguards. These findings strengthen overall code quality and reduce operational risks arising from initialization errors, state bloat, and incorrect event emissions.

Audit Scope

- **Repository:** [pact-governance](#)
- **Commit Hash:** [c4489c325b701dff6ec8d42b979f17f104c4d557](#)
- **Files/Modules in Scope:**
 - governance.move
 - vesting.move
 - airdrop.move
 - distribution.move
 - vetoken.move
 - ve_token.move
 - ve_token_public_api.move
- **Fix Review Final Commit Hash**
 - * [0ef206818b3024f2371abae552a83d1033ea328d](#)

2. Assumptions and Considerations

Audit Assumptions

I. Aptos Framework Security: The audit assumes that the underlying Aptos blockchain and its core framework modules (AptosFramework, AptosTokenObjects) are secure and functioning as documented.

II. Move Language Semantics: The audit assumes Move language safety guarantees hold, including resource linearity, memory safety, and type safety enforced by the Move VM.

III. Deployment Configuration: The audit assumes proper deployment procedures will be followed, including correct address configuration, proper initialization of modules, and secure management of deployment keys.

IV. Time Source Reliability: The protocol heavily relies on `timestamp::now_seconds()` for time-based calculations. The audit assumes the Aptos blockchain provides accurate and monotonically increasing timestamps.

V. External Token Standards: The audit assumes fungible assets and token objects used with the protocol conform to Aptos standards and behave predictably.

VI. Mathematical Operations: The audit assumes no integer overflow/underflow due to Move's built-in safety checks, though some division-by-zero risks were identified.

VII. Initialization Signer Authority: The audit identified that initialization signers must match the `@voting` address to prevent deployment footguns. The audit assumes proper verification will be implemented before deployment.

Centralization Risks

* Admin can amend vesting schedules mid-stream, potentially changing end times and affecting issuance rates for active vesting periods.

* Admin can update user schedules arbitrarily, potentially switching users to slower schedules after they've claimed significant amounts.

* Governance can retroactively change auction durations mid-flight, ending or extending ongoing auctions unexpectedly.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product’s reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds,** and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

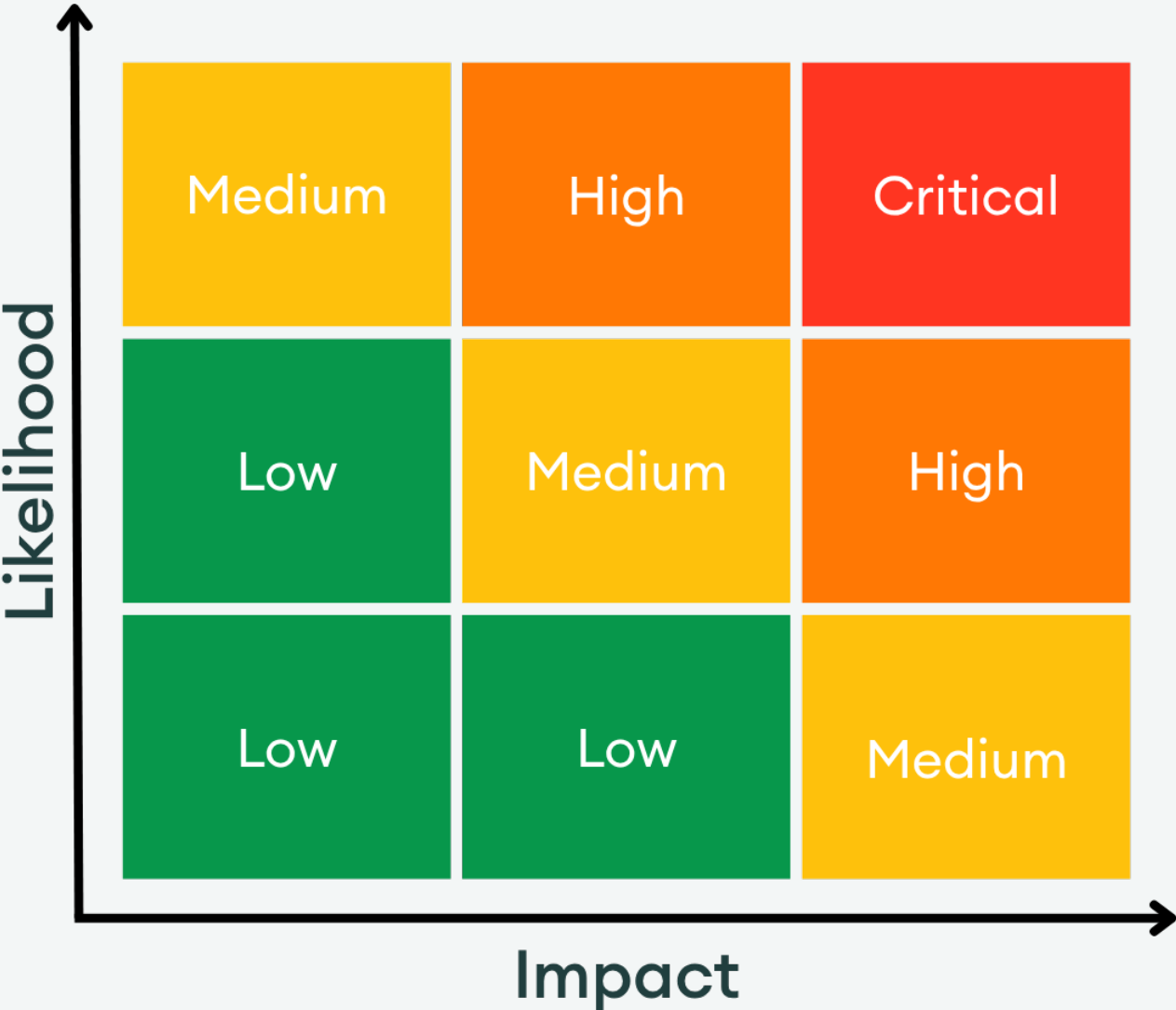
- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

M01: Incorrect issuance rate when `ammend_vesting_end` is used before the cliff

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Medium

Location:

<https://github.com/Lucid-Fi/pact-governance/blob/staging/contract/sources/vesting.move#L442>

> _vesting.move

```
440:     };  
441:  
442:     let start_time = math64::max(self.start_time_seconds, self.mutation_record.last_updated_seconds);  
443:     if (start_time >= timestamp_seconds) {  
444:         return self.mutation_record.last_vesting_progress;
```

Description:

Let's consider an example:

The vesting schedule for one user is the following: starts at 0, ends at 100, has cliff at 50, total tokens vested are 100. This effectively mean that the user will receive nothing till 50s, and then the issuance will be 1 token per 1 second.

Let's say `ammend_vesting_end` vesting end is called at 40s and sets the new end to the vesting at 200s. So effectively doubling the vesting period. This mean that the user should receive at cliff time only half as he did before and then the issuance rate should be 1 token per 2 seconds. Or 0.5 tokens pers second.

What happens instead is that at cliff, the user will receive just tokens just for 50s-40s=10s instead of for 50s. And the issuance rate after this will be 100 tokens per 200s-40s=160s. Which is 0.625 tokens per second. So at cliff instead of receiving 25 tokens he only receives 0.625*10=6.25 tokens and then the issuance rate will be bigger than intended (0.625 per second instead of 0.5 tokens per second).

Recommendation:

One way, maybe not the cleanest, would be to set the `last_updated_seconds` to `start_time_seconds` if the `ammend_vesting_end` is called before the cliff. Or just deny amending before the cliff.

Developer Response:

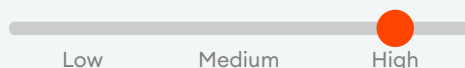
Ignored cliff when calculating mutation record on schedule amendment.

Fixed in commit a8d1499323b58b6d8fe6b75ffc82979c73d81b96

M02: Locks are not processed in order as expected

Status:

Resolved

Impact:**Likelihood:****Severity:**

Medium

Location: https://github.com/AdevarLabs/pact-governance-reference/blob/ce3cfc97d9e2e523475d8b18662f8eef5c94e40c/contract/sources/ve_token.move#L341-L371

```
> _ve_token.move
339:     let remaining_amount = unlock_amount;
340:     let user_epochs = user_locks.keys();
341:     for (i in 0..user_epochs.length()) {
342:         let epoch = user_epochs.borrow(i);
343:         if (!user_locks.contains_key(epoch)) {
344:             continue;
345:         };
346:
347:         // Update the lock if it has enough amount to cover the unlock or else remove it.
348:         updated_lock_epochs.push_back(*epoch);
349:         let locked_amount = user_locks.borrow_mut(epoch);
350:         let amount_used =
351:             if (*locked_amount > remaining_amount) {
352:                 *locked_amount -= remaining_amount;
353:                 remaining_amount
354:             } else {
355:                 let locked_amount = *locked_amount;
356:                 user_locks.remove(epoch);
357:                 locked_amount
358:             };
359:
360:         remaining_amount -= amount_used;
361:         let global_locked_amount = global_locks.borrow_mut(epoch);
362:         *global_locked_amount -= amount_used;
363:         if (*global_locked_amount == 0) {
364:             global_locks.remove(epoch);
365:         };
366:
367:         // Break early to save gas if we have unlocked the entire amount.
368:         if (remaining_amount == 0) {
369:             break;
370:         };
371:     };
372:     // User does not have enough locked tokens to cover the request amount.
373:     assert!(remaining_amount == 0, ECANNOT_UNLOCK_MORE_THAN_TOTAL_LOCKED);
```

Description:

SimpleMap doesn't preserve order (`remove` uses `swap-remove`), so iterating `user_locks.keys()` won't give "earliest first." The comment says "Deduct ... from the earliest," but in practice it

burns locks in whatever internal order the map has at the time, potentially consuming newer epochs first and letting older, higher multiplier locks stay, which changes voting power semantics.

Recommendation:

If deterministic oldest-first is needed, sort the keys or repeatedly pick the minimum epoch.

Developer Response:

Used OrderedMap instead of SimpleMap.

Fixed in commit `c89c2c6301306b08d7e1ee7606d65c52714253c1`

M03: Unlock history can grow unbounded

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Medium

Location: https://github.com/AdevarLabs/pact-governance-reference/blob/ce3cfc97d9e2e523475d8b18662f8eef5c94e40c/contract/sources/ve_token.move#L182-L194

```
>_ve_token.move
180:     let remaining_unlocks = vector[];
181:     let total_withdraw = 0;
182:     user_unlocks.for_each_ref(
183:         |unlock: &TokenUnlock| {
184:             if (unlock.unlocks_at <= timestamp::now_seconds()) {
185:                 total_withdraw += unlock.unlock_amount;
186:             } else {
187:                 // User cannot withdraw the unlock amount yet.
188:                 remaining_unlocks.push_back(*unlock);
189:             };
190:         }
191:     );
192:
193:     (total_withdraw, remaining_unlocks)
194: }
195:
196: #[view]
```

Description:

The `unlock` function allows a user to create a new `TokenUnlock` record every time it's called. These records are added to a vector in the `user_unlocks` table. The `withdraw` function, in turn, must iterate over this entire vector to calculate the total withdrawable amount.

A constant `MAX_WITHDRAW_RECORDS` is defined at the top of the file, but it is never used or enforced. There is no limit to the number of unlock records a user can accumulate.

A user can call `unlock` hundreds of times with tiny amounts. This will create a very large vector of `TokenUnlock` records. When the user later calls `withdraw`, the transaction will fail because the gas required to loop over the huge vector will exceed the block gas limit. This will permanently lock the user's funds, as they will be unable to ever successfully call `withdraw`.

Recommendation:

Enforce the limit in the unlock function. Before adding a new `TokenUnlock` record, check the length of the vector and abort if it has reached `MAX_WITHDRAW_RECORDS`. Additionally and optionally, add a `merge_withdrawals` entry function that a user can call to consolidate all their currently withdrawable amounts into a single `TokenUnlock` record, freeing up space for new unlock requests.

Developer Response:

Added merge and cap for unlock entries.

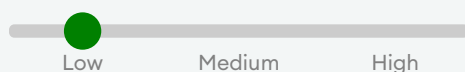
Fixed in commit 9d3d5438e6436e9753ac0dbe6d8625b7f5bf5579

L01: State bloat via unauthenticated vesting initialization

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location:

<https://github.com/Lucid-Fi/pact-governance/blob/staging/contract/sources/vesting.move#L329>

> _vesting.move

```
327:     }
328:
329:     public fun deposit(fungible_asset: fungible_asset::FungibleAsset, wallet: address)
    {
330:         assert!(vote_token::is_vote_token(&fungible_asset), ENOT_VOTE_TOKEN);
331:         let manager = manager_mut();
```

Description:

`deposit` is public and auto-creates a vesting state for any wallet if none exists, calling `object::create_sticky_object(@voting)` and inserting into `vesting_states` even for `amount == 0`. An attacker can spam arbitrary wallets with zero-value deposits, forcing unbounded sticky objects and table entries under `@voting`. Besides the state bloating, an attacker can make any wallet look like it is part of a team that has skin-in-the-game because the vesting object is created by default under the investor vesting schedule.

Recommendation:

Require `amount > 0` and also `require signer == wallet` or an authorized initializer (consider making deposit an `entry` and gating it by an allowlist/capability).

Developer Response:

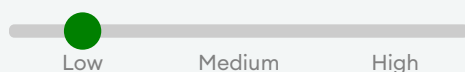
Added authentication for vesting deposit.
Fixed in commit `e7175b328f97fd16d4c3e5ecd3cd6309e2f335e2`

L02: Integer underflow leads to DoS in `vesting::claim`

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location:

<https://github.com/Lucid-Fi/pact-governance/blob/staging/contract/sources/vesting.move#L432>

> _vesting.move

```
430: let vesting_progress = schedule.vested_as_of(timestamp_seconds);
431: let vested_amount = math128::mul_div(vesting_progress, vesting_state.total_amo
unt_to_vest as u128, VESTING_PROGRESS_DENOMINATOR) as u64;
432: let claimable_amount = vested_amount - vesting_state.total_amount_claimed;
433:
434: (vested_amount, claimable_amount)
```

Description:

The line will cause an integer underflow, leading to a transaction abort, under a specific scenario:

1. A user is on a vesting schedule (Schedule A) and claims a significant amount of their tokens.
2. An admin then uses the `update_user_schedule` function to switch the user to a much slower vesting schedule (Schedule B).
3. Immediately after the switch, the `vested_amount` calculated using the new, slower Schedule B could be less than the `total_amount_claimed` that the user has already withdrawn under the old Schedule A.
4. When this happens, the subtraction `vested_amount - total_amount_claimed` attempts to compute a negative number, resulting in an arithmetic underflow that aborts the transaction.

This is especially problematic because it will cause the `#[view]` function `claimable_by_user` to fail, and it will block the user from using the `claim` function until the new vesting schedule "catches up" to the amount they have already claimed.

Recommendation:

The fix is to ensure that `claimable_amount` cannot be negative. If the `vested_amount` is less than what has already been claimed, the amount currently claimable should be 0.

Developer Response:

Checked for underflow in vesting progress.

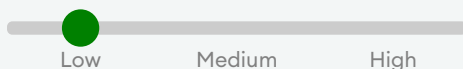
Fixed in commit `0c6b451985edd248c05b3ec7e16545c88403e6d4`

L03: RewardsClaimed event is always emitted with epochs_processed==0

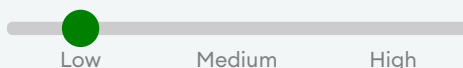
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: <https://github.com/Lucid-Fi/pact-governance/blob/staging/contract/sources/distribution.move#L313>

> _distribution.move

```
311:         user: user_addr,  
312:         amount_claimed: total_claimable,  
313:         epochs_processed  
314:     }  
315: );
```

Description:

The function calculates `total_claimable`, updates user and epoch states, and then performs the token transfer.

The issue is that the `epochs_processed` variable is initialized to 0 and never updated, but it is emitted in the `RewardsClaimed` event. This is not a security flaw but a bug that will lead to incorrect event data.

Recommendation:

Update `epochs_processed` in the loop before using it in the event.

Developer Response:

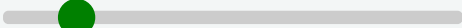
Used epoch keys length for `epochs_processed`.

Fixed in commit 6f6efb2ab4a729156c572b027fa52c951bd6be5d

L04: The loop in `unsynced_user_epoch_progress` can run out of gas

Status:

Resolved

Impact: Low Medium High**Likelihood:** Low Medium High**Severity:**

Low

Location: <https://github.com/Lucid-Fi/pact-governance/blob/staging/contract/sources/distribution.move#L373-L406>

>_distribution.move

```
371:     epoch_keys.reverse();
372:
373:     for (i in 0..epoch_keys.length()) {
374:         let epoch = epoch_keys[i];
375:         if (epoch <= last_synced_epoch) {
376:             break;
377:         };
378:         if (epoch == current_epoch) {
379:             continue;
380:         };
381:
382:         let epoch_state = state.epoch_rewards.borrow(&epoch);
383:         let user_voting_power = ve_token::voting_power_at(user_address, epoch);
384:         let voting_power_denominator = if (epoch_state.voting_power_denominator >
0) {
385:             epoch_state.voting_power_denominator
386:         } else {
387:             voting_power_denominator_at(epoch)
388:         };
389:
390:         let user_claimable = if (voting_power_denominator > 0 && user_voting_power
> 0) {
391:             math128::mul_div(
392:                 epoch_state.total_available as u128,
393:                 user_voting_power,
394:                 voting_power_denominator
395:             ) as u64
396:         } else {
397:             0
398:         };
399:
400:         if (user_claimable > 0) {
401:             unsynced_epochs.push_back(UnsyncedUserEpochProgress {
402:                 epoch,
403:                 total_claimable: user_claimable
404:             });
405:         };
406:     };
407:
408:     unsynced_epochs
```

Description:

`sync_user_claim_state_internal` calls `unsynced_user_epoch_progress` which walks all epochs between `last_epoch_snapshot+1` and the latest finalized reward. If a user hasn't synced for many epochs, this loop can blow up in iterations/gas. `claim_from_epochs` already caps work via `max_epochs`, but `sync_user_claim_state_internal` has no such guard.

However the likelihood of this being a real problem is very low, because an epoch is configured as 7 days. A year would contain around 52 epochs, so the loop would be able to process an user that hasn't sync for a couple of years.

Recommendation:

Add a configurable/passed-in `max_epochs_to_sync` (or slice the epoch key vector) to process epochs in batches, returning a signal when more remain. Expose a public entry that takes the max and increments `last_epoch_snapshot` as it goes, mirroring the `max_epochs` pattern in `claim_from_epochs`.

Developer Response:

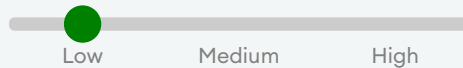
Added entry method to sync user claim states with capped iterations.
Fixed in commit 0ef206818b3024f2371abae552a83d1033ea328d

L05: `global_vesting_amounts` is never updated so `global_vested_by_schedule` is wrong

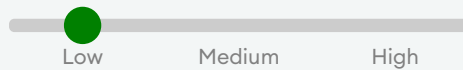
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: <https://github.com/AdevarLabs/pact-governance-reference/blob/ba83a6947b3d5d2d2aa02cfe1ca0ac3791b9925b/contract/sources/vesting.move#L78-L93>

> _vesting.move

```
76:     }
77:
78:     struct VestingManagement has key {
79:         /// The vesting schedules for different groups of wallets
80:         vesting_schedules: ordered_map::OrderedMap<vector<u8>, VestingSchedule>,
81:         /// The total amount of vested tokens for each schedule
82:         global_vesting_amounts: ordered_map::OrderedMap<vector<u8>, u128>,
83:         /// The states of the vesting for each user
84:         vesting_states: table::Table<address, VestingState>,
85:         /// The admins of the vesting system
86:         admins: ordered_map::OrderedMap<address, bool>,
87:         /// The wallets that can add new users and set their initial schedule
88:         initializers: ordered_map::OrderedMap<address, bool>,
89:         /// The default schedule key for new users
90:         default_schedule_key: vector<u8>,
91:         /// Tracks global vested locks
92:         global_vested_locks: ve_token::LockRecords
93:     }
94:
95:     #[event]
```

Description:

the following struct has field `global_vesting_amounts`

RUST

```
struct VestingManagement has key {
    vesting_schedules: ordered_map::OrderedMap<vector<u8>, VestingSchedule>,
    global_vesting_amounts: ordered_map::OrderedMap<vector<u8>, u128>,
    ...
}

#[view]
public fun global_vested_by_schedule(schedule_key: vector<u8>): u128 {
    let manager = manager();
    let schedule = manager.vesting_schedules.borrow(&schedule_key);
    let now = timestamp::now_seconds();
    let vested_amount = schedule.vested_as_of(now);
}
```


... continued

RUST

```
let total_vesting_amount = *manager.global_vesting_amounts.borrow(&schedule_key);  
  
math128::mul_div(vested_amount, total_vesting_amount, VESTING_PROGRESS_DENOMINATOR)  
}
```

global_vesting_amounts is initialized to zeros in init_module but no where in the code we update it.

so the `global_vested_by_schedule` is effectively useless.

Recommendation:

Increment global_vesting_amounts[schedule_key] whenever a user with that schedule deposits

Developer Response:

Updated global_vesting_amounts alongside deposits.

Fixed in commit 266557908eea5b2a7b1fcec0fcc11ecbc6d7726d

L06: Airdrop can be created already expired

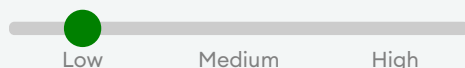
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: <https://github.com/AdevarLabs/pact-governance-reference/blob/ba83a6947b3d5d2d2aa02cfe1ca0ac3791b9925b/airdrop-contracts/sources/airdrop.move#L71-L95>

> _airdrop.move

```
69:     }
70:
71:     public fun create_airdrop(
72:         creator: &signer,
73:         merkle_root: vector<u8>,
74:         token: Object<Metadata>,
75:         admin: address,
76:         expiration_timestamp: u64
77:     ): Object<AirdropConfig> {
78:         let creator_addr = signer::address_of(creator);
79:         let constructor_ref = &object::create_sticky_object(creator_addr);
80:         let object_signer = &object::generate_signer(constructor_ref);
81:         let airdrop_address = signer::address_of(object_signer);
82:
83:         move_to(
84:             object_signer,
85:             AirdropConfig {
86:                 merkle_root,
87:                 token,
88:                 admin,
89:                 expiration_timestamp,
90:                 claimed: table::new(),
91:                 store_extend_ref: object::generate_extend_ref(constructor_ref),
92:                 total_deposited: 0,
93:                 total_claimed: aggregator_v2::create_unbounded_aggregator()
94:             }
95:         );
96:
97:         event::emit(AirdropCreated {
```

Description:

signature for the `create_airdrop` function looks like following

```
public fun create_airdrop(
    creator: &signer,
    merkle_root: vector<u8>,
    token: Object<Metadata>,
```

RUST

... continued**RUST**

```
admin: address,  
expiration_timestamp: u64  
)
```

but there is no validation like following in this create function

RUST

```
assert!(expiration_timestamp > timestamp::now_seconds(), EAIRDROP_EXPIRED);
```

So you can create an airdrop where:

- expiration_timestamp <= now_seconds()
- Every claim call immediately aborts with EAIRDROP_EXPIRED.

Recommendation:

add a check that validates the expiry timestamp is in future.

Developer Response:

Enforced expiration time is in the future.

Fixed in commit a232be9f49c103b552663af1f11bca3e4e76ce35

L07: Admin can withdraw remaining even when the airdrop has not expired.

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Low

Location: <https://github.com/AdevarLabs/pact-governance-reference/blob/ba83a6947b3d5d2d2a02cfelca0ac3791b9925b/airdrop-contracts/sources/airdrop.move#L249-L262>

>_ airdrop.move

```
247:     }
248:
249:     public entry fun withdraw_remaining(
250:         admin: &signer,
251:         airdrop: Object<AirdropConfig>,
252:         amount: u64
253:     ) acquires AirdropConfig {
254:         let admin_addr = signer::address_of(admin);
255:         let config = borrow_config(&airdrop);
256:         assert!(admin_addr == config.admin, ENOT_ADMIN);
257:
258:         let store_signer = &object::generate_signer_for_extending(&config.store_extend_ref);
259:         primary_fungible_store::transfer(store_signer, config.token, admin_addr, amount);
260:
261:         event::emit(TokensWithdrawn { admin: admin_addr, amount });
262:     }
263:
264:     inline fun borrow_config(airdrop: &Object<AirdropConfig>): &AirdropConfig {
```

Description:

There is no check that the airdrop has expired before withdraw_remaining is allowed.

RUST

```
public entry fun withdraw_remaining(
    admin: &signer,
    airdrop: Object<AirdropConfig>,
    amount: u64
) acquires AirdropConfig {
    let admin_addr = signer::address_of(admin);
    let config = borrow_config(&airdrop);
    assert!(admin_addr == config.admin, ENOT_ADMIN);

    let store_signer = &object::generate_signer_for_extending(&config.store_extend_ref);
    primary_fungible_store::transfer(store_signer, config.token, admin_addr, amount);

    event::emit(TokensWithdrawn { admin: admin_addr, amount });
}
```

... continued

RUST

}

After that, all claim calls will revert during `primary_fungible_store::transfer` due to insufficient balance, but users pay gas and never get tokens.

Recommendation:

add an expiry check

RUST

```
assert!(timestamp::now_seconds() >= config.expiration_timestamp, EAIRDROP_EXPIRED);
```

Developer Response:

Required airdrop expiration for admin to withdraw tokens.

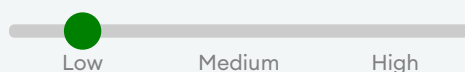
Fixed in commit `b290f3e5fbc81a145cfcf2ecd8b81a96a67629e5`

L08: Zero-amount unlocks are accepted

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/AdevarLabs/pact-governance-reference/blob/ce3cfc97d9e2e523475d8b18662f8eef5c94e40c/contract/sources/ve_token.move#L325-L392

> _ve_token.move

```
323:     }
324:
325:     public entry fun unlock(user: &signer, unlock_amount: u64) acquires VeTokenManagement {
326:         // Always compact locks first to prevent user locks from growing too large.
327:         compact_user_locks(user);
328:
329:         let user_addr = signer::address_of(user);
330:         let config = &mut VeTokenManagement[@voting];
331:         let user_lock_records = config.user_locks.borrow_mut(user_addr);
332:         let user_locks = &mut user_lock_records.locks;
333:         assert!(user_locks.length() > 0, ENO_USER_LOCKS);
334:         let global_locks = &mut config.global_locks.locks;
335:         assert!(global_locks.length() > 0, EINVAL_STATE_NO_GLOBAL_LOCKS);
336:
337:         // Deduct the amount from locks starting from the earliest. If the locks are insufficient, abort.
338:         let updated_lock_epochs = vector[];
339:         let remaining_amount = unlock_amount;
340:         let user_epochs = user_locks.keys();
341:         for (i in 0..user_epochs.length()) {
342:             let epoch = user_epochs.borrow(i);
343:             if (!user_locks.contains_key(epoch)) {
344:                 continue;
345:             };
346:
347:             // Update the lock if it has enough amount to cover the unlock or else remove it.
348:             updated_lock_epochs.push_back(*epoch);
349:             let locked_amount = user_locks.borrow_mut(epoch);
350:             let amount_used =
351:                 if (*locked_amount > remaining_amount) {
352:                     *locked_amount -= remaining_amount;
353:                     remaining_amount
354:                 } else {
355:                     let locked_amount = *locked_amount;
356:                     user_locks.remove(epoch);
357:                     locked_amount
358:                 };
359:
```

```
>_ve_token.move
```

```
360:         remaining_amount -= amount_used;
361:         let global_locked_amount = global_locks.borrow_mut(epoch);
362:         *global_locked_amount -= amount_used;
363:         if (*global_locked_amount == 0) {
364:             global_locks.remove(epoch);
365:         };
366:
367:         // Break early to save gas if we have unlocked the entire amount.
368:         if (remaining_amount == 0) {
369:             break;
370:         };
371:     };
372:     // User does not have enough locked tokens to cover the request amount.
373:     assert!(remaining_amount == 0, ECANNOT_UNLOCK_MORE_THAN_TOTAL_LOCKED);
374:
375:     // Update the unlock records for the user.
376:     if (!config.user_unlocks.contains(user_addr)) {
377:         config.user_unlocks.add(user_addr, vector[]);
378:     };
379:     let unlocks = config.user_unlocks.borrow_mut(user_addr);
380:     let unlocks_at = timestamp::now_seconds() + config.unlock_delay;
381:     unlocks.push_back(TokenUnlock { unlock_amount, unlocks_at });
382:
383:     event::emit(
384:         UnlockEvent {
385:             user: user_addr,
386:             unlock_amount,
387:             unlocks_at,
388:             requested_epoch: current_epoch(),
389:             updated_lock_epochs
390:         }
391:     );
392: }
393:
394: public entry fun withdraw(user: &signer) acquires VeTokenManagement {
```

Description:

The function doesn't assert `unlock_amount > 0`, so users can add zero-value `TokenUnlock` entries, compounding the state-growth/gas issue.

Recommendation:

Add a non-zero check.

Developer Response:

Aborted on unlock of zero.

Fixed in commit 3486f538470def2f6614465e1db792549ab8e9f3

5. Enhancement Opportunities

E01: Enforce more checks in `ammend_vesting_end`

Location: <https://github.com/Lucid-Fi/pact-governance/blob/staging/contract/sources/vesting.move#L456-L457>

```
> _vesting.move
454:
455:     fun ammend_vesting_end(self: &mut VestingSchedule, new_vest_end_time_seconds: u64,
456:         current_time_seconds: u64) {
457:         assert!(new_vest_end_time_seconds > current_time_seconds, EVEST_END_ALREADY_PA
458:             SSED);
459:         assert!(new_vest_end_time_seconds > self.mutation_record.last_updated_seconds,
460:             EVEST_END_BEFORE_LAST_UPDATED);
461:
462:         let current_progress = self.vested_as_of(current_time_seconds);
```

Description:

This function should check that the new end time is at least greater than it's start time. And it should not be applied if the vesting already ended.

Potential Benefit:

More reliance on the code, more resilient to input mistakes.

Recommendation:

Add more checks to assure that the new end time is strictly greater than start time. Maybe even greater than the current end time (to always make the release slower instead of faster to protect the community, but this depends on the business logic and intended use case). Also make sure this is not applied to a vesting schedule that already ended.

Developer Response:

Added two assertions to enforce validation checks: one to ensure the new vesting end time is after the start time (`EVEST_END_BEFORE_START`), and another to verify the vesting hasn't already ended (`EALREADY_FULLY_VESTED`).

Fixed in commit `0c65be034ee91b3a804184b864a73eacb023d0`

E02: Enforce more checks in `update_user_schedule`

Location: <https://github.com/Lucid-Fi/pact-governance/blob/staging/contract/sources/vesting.move#L313-L317>

> _vesting.move

```
311:
312:     public entry fun update_user_schedule(signer: &signer, user: address, new_schedule
    _key: vector<u8>) {
313:         assert!(is_admin(signer::address_of(signer)), ENOT_ADMIN);
314:         let manager = manager_mut();
315:         let vesting_state = manager.vesting_states.borrow_mut(user);
316:         let old_schedule_key = vesting_state.vesting_schedule_key;
317:         vesting_state.vesting_schedule_key = new_schedule_key;
318:
319:         event::emit(
```

Description:

There's no check to see if the user that is intended to be updated exists. Moreover `update_user_schedule` does not verify the new key exists in `vesting_schedules`.

Potential Benefit:

Have a clear error message in case of mistakes when admin is calling this function.

Recommendation:

Check if the user exists. Check if the new `vesting_schedule_key` is known.

Developer Response:

Added assertions to verify that the new schedule key exists (`ESCHEDULE_NOT_FOUND`) and the user's vesting state exists (`EVESTING_STATE_NOT_FOUND`) before updating.
Fixed in commit 08ce90635d69aaf5187324adefbdd601b4d86760

E03: withdraw_remaining ignores EINSUFFICIENT_BALANCE and balance checks

Location: <https://github.com/AdevarLabs/pact-governance-reference/blob/ba83a6947b3d5d2d2aa02cfe1ca0ac3791b9925b/airdrop-contracts/sources/airdrop.move#L258-L259>

```
> _airdrop.move
256:      assert!(admin_addr == config.admin, ENOT_ADMIN);
257:
258:      let store_signer = &object::generate_signer_for_extending(&config.store_extend
    _ref);
259:      primary_fungible_store::transfer(store_signer, config.token, admin_addr, amount);
260:
261:      event::emit(TokensWithdrawn { admin: admin_addr, amount });
```

Description:

following error is defined in the code:

```
/// Insufficient balance to withdraw
const EINSUFFICIENT_BALANCE: u64 = 5;
```

RUST

but it's never used.

`withdraw_remaining` just calls

```
primary_fungible_store::transfer(store_signer, config.token, admin_addr, amount);
```

RUST

If `amount > airdrop_balance`, the function just aborts with whatever internal error `primary_fungible_store` uses.

Potential Benefit:

better error handling and code readability.

Recommendation:

use the declared error code.

Developer Response:

Used internal error code for insufficient balance on withdraw.

Fixed in commit `8b5f5dbde4eachb200d68ac0fae90763b7cf86b7d`

E04: Fragile unlock logic

Location: https://github.com/AdevarLabs/pact-governance-reference/blob/ce3cfc97d9e2e523475d8b18662f8eef5c94e40c/contract/sources/ve_token.move#L325-L392

> _ve_token.move

```
323:     }
324:
325:     public entry fun unlock(user: &signer, unlock_amount: u64) acquires VeTokenManagem
ent {
326:         // Always compact locks first to prevent user locks from growing too large.
327:         compact_user_locks(user);
328:
329:         let user_addr = signer::address_of(user);
330:         let config = &mut VeTokenManagement[@voting];
331:         let user_lock_records = config.user_locks.borrow_mut(user_addr);
332:         let user_locks = &mut user_lock_records.locks;
333:         assert!(user_locks.length() > 0, ENO_USER_LOCKS);
334:         let global_locks = &mut config.global_locks.locks;
335:         assert!(global_locks.length() > 0, INVALID_STATE_NO_GLOBAL_LOCKS);
336:
337:         // Deduct the amount from locks starting from the earliest. If the locks are i
nsufficient, abort.
338:         let updated_lock_epochs = vector[];
339:         let remaining_amount = unlock_amount;
340:         let user_epochs = user_locks.keys();
341:         for (i in 0..user_epochs.length()) {
342:             let epoch = user_epochs.borrow(i);
343:             if (!user_locks.contains_key(epoch)) {
344:                 continue;
345:             };
346:
347:             // Update the lock if it has enough amount to cover the unlock or else rem
ove it.
348:             updated_lock_epochs.push_back(*epoch);
349:             let locked_amount = user_locks.borrow_mut(epoch);
350:             let amount_used =
351:                 if (*locked_amount > remaining_amount) {
352:                     *locked_amount -= remaining_amount;
353:                     remaining_amount
354:                 } else {
355:                     let locked_amount = *locked_amount;
356:                     user_locks.remove(epoch);
357:                     locked_amount
358:                 };
359:
360:             remaining_amount -= amount_used;
361:             let global_locked_amount = global_locks.borrow_mut(epoch);
362:             *global_locked_amount -= amount_used;
363:             if (*global_locked_amount == 0) {
364:                 global_locks.remove(epoch);
365:             };
366:
367:             // Break early to save gas if we have unlocked the entire amount.
368:             if (remaining_amount == 0) {
369:                 break;
370:             };
371:         };
```

>_ve_token.move

```
372:         // User does not have enough locked tokens to cover the request amount.
373:         assert!(remaining_amount == 0, ECANNOT_UNLOCK_MORE_THAN_TOTAL_LOCKED);
374:
375:         // Update the unlock records for the user.
376:         if (!config.user_unlocks.contains(user_addr)) {
377:             config.user_unlocks.add(user_addr, vector[]);
378:         };
379:         let unlocks = config.user_unlocks.borrow_mut(user_addr);
380:         let unlocks_at = timestamp::now_seconds() + config.unlock_delay;
381:         unlocks.push_back(TokenUnlock { unlock_amount, unlocks_at });
382:
383:         event::emit(
384:             UnlockEvent {
385:                 user: user_addr,
386:                 unlock_amount,
387:                 unlocks_at,
388:                 requested_epoch: current_epoch(),
389:                 updated_lock_epochs
390:             }
391:         );
392:     }
393:
394:     public entry fun withdraw(user: &signer) acquires VeTokenManagement {
```

Description:

The **unlock** function iterates through a snapshot of the user's lock epochs while simultaneously modifying the underlying **SimpleMap** of locks (by reducing amounts or removing entries). This pattern (modifying a collection while iterating over a snapshot of its keys) is notoriously complex and prone to subtle bugs, especially if the code is changed later. While the function is correctly written, its fragility poses a maintenance risk.

Potential Benefit:

Refactoring this would provide the code a safer and more readable pattern.

Recommendation:

Refactor this part, a **while** loop that always processes the first/earliest remaining lock until **remaining_amount** is zero would be more robust and easier to understand.

Developer Response:

The unlock logic now collects epochs to remove in a separate vector and deletes them **after** the loop, instead of removing locks while iterating over the map's keys.

Fixed in commit c89c2c6301306b08d7e1ee7606d65c52714253c1

E05: Incorrect error code in `get_upgrader_signer`

Location: <https://github.com/AdevarLabs/pact-governance-reference/blob/ce3cfc97d9e2e523475d8b18662f8eef5c94e40c/contract/sources/governance.move#L425-L429>

```
> _governance.move
423:     resource_request: ResourceRequest
424: ): signer acquires GovernanceParameters {
425:     assert!(
426:         resource_request.type() == MODULE_UPGRADE,
427:         EINVALID_RESOURCE_REQUEST_FOR_GOV_PARAMS
428:     );
429:     object::generate_signer_for_extending(&GovernanceParameters[@voting].upgrader)
430: }
431:
```

Description:

The logic is correct, but if the assertion fails, it will emit a misleading error reason (`EINVALID_RESOURCE_REQUEST_FOR_GOV_PARAMS` instead of `EINVALID_RESOURCE_REQUEST_FOR_UPGRADE`).

Potential Benefit:

Fixing this would improve debugging in some cases.

Recommendation:

Replace the error constant with the correct one, `EINVALID_RESOURCE_REQUEST_FOR_UPGRADE`

Developer Response:

Used correct error code for `get_upgrader_signer`.

Fixed in commit 221ed00fffa59a303bb3d656a695945a103128e3

E06: Assert before voting power truncation (u128 to u64)

Location: <https://github.com/AdevarLabs/pact-governance-reference/blob/ce3cfc97d9e2e523475d8b18662f8eef5c94e40c/contract/sources/governance.move#L310-L316>

```
> _governance.move
308:     let voting_power =
309:         ve_token::voting_power_at(voter_address, ve_token::current_epoch());
310:     voting::vote<GovernanceProposal>(
311:         &GovernanceProposal {},
312:         @voting,
313:         proposal_id,
314:         voting_power as u64,
315:         should_pass
316:     );
317:
318:     event::emit(
```

Description:

This is a cast to **u64** because **aptos_framework::voting::vote** only accepts **u64** weights. There shouldn't be a problem because the bonus factor is capped at 4x (52 epochs * 1/13 multiplier), so in order for an actual truncation one would need $>4.61e18$ raw units ($\sim 46B.1$ tokens with 8 decimals) locked to overflow.

Potential Benefit:

Improves the reliability in the code and makes sure the voting power is never truncated.

Recommendation:

Add a guard before casting, eg. **assert!(voting_power <= u64::MAX)**.

Developer Response:

Asserted voting power fits in u64.

Fixed in commit 564703789f2b4fda71e1e795337abbe4091aa5a4

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Aptos Move frameworks and modules. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure every Move package and entry function operates securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Aptos Move package documentation, intended functionality, and resource design. We meticulously map out module interactions, transactional entry points, and global storage layout. Special attention is given to understanding how your code interfaces with core Aptos framework modules, such as `aptos_framework::coin`, `aptos_framework::account`, and `aptos_framework::timestamp`. We also review integrations with external Move packages and ensure all abilities, arguments, and return values are validated correctly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Aptos' Move VM and resource-oriented execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Move-specific issues, including but not limited to:

- Unauthorized resource publication or key account access

- Improper signer capability or `&signer;` usage
- Misconfigured `friend` declarations and module visibility
- Unsafe global storage writes or missing resource invariants
- Failure to enforce ability constraints (`key`, `store`, `drop`, `copy`)
- Risks stemming from reconfiguration hooks, event emission, or aggregator usage

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Move modules. This involves a comprehensive line-by-line inspection of source code, focusing on common Aptos vulnerabilities including, but not limited to:

- Missing or insufficient signer and address authorization checks
- Incorrect handling of capability storage, delegation, or revocation
- Unsafe use of table/aggregator APIs or unchecked resource destruction
- Arithmetic errors in on-chain invariants or interest calculations
- Incorrect event sequencing or metadata emissions
- Logic flaws in `move_to/move_from` flows or resource initialization
- Missing abort conditions on critical preconditions and edge-case validation
- Potential denial-of-service vectors tied to gas budgeting, storage growth, or reconfiguration

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Aptos Move best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Aptos Move ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.